# Predicting Exoplanet Equilibrium Temperature with Linear Regression

*Task: Planetary Devastation | Data: NASA Exoplanet Archive (PSCompPars) | NumPy and Pandas only*

## 1. Data

The PSCompPars table has 6,451 confirmed planets and 84 columns. A lot of the columns are mostly empty (error bounds, spectral type), so calling dropna() on the whole table would throw away most of the catalogue. I only dropped rows where the target, pl_eqt, was missing (about 8.4%). That left 5,911 planets.

I used eight predictors: pl_rade, pl_bmasse, pl_orbper, pl_orbsmax, st_teff, st_rad, st_mass and sy_dist. Each has under 5% missing values in the filtered data (pl_orbsmax is the worst at 4.48%).

## 2. What the EDA showed

pl_eqt is right-skewed. Most planets sit between 300 K and 2,000 K, with a tail of very hot gas giants above 3,000 K.

Orbital period and semi-major axis had almost no linear correlation with temperature in raw form (r = -0.01 and 0.02), because a few very long-period planets stretch the scale. After taking log10 of both, the correlations became r = -0.66 for log_orbper and r = -0.48 for log_orbsmax. So the log transform was necessary. Among the raw features, st_teff had the strongest correlation with temperature (r = 0.32). The two orbital features are also nearly collinear (r = 1.00), which follows from Kepler's third law.

As a sanity check I computed the blackbody estimate Teq = Teff x sqrt(R_star / 2a). It correlated with pl_eqt at r = 0.93, so the physics holds up in the data. I did not use it as a feature in the final model.

## 3. Preprocessing and imputation

Missing values were handled in two steps. First, rows with no pl_eqt were removed, since the target cannot be imputed. Second, I checked how much of each predictor was missing in the remaining 5,911 rows before choosing a strategy:

| Feature | Missing | Feature | Missing |
|---|---|---|---|
| pl_orbsmax | 4.48% | pl_orbper | 1.18% |
| pl_rade | 0.66% | pl_bmasse | 0.44% |
| sy_dist | 0.37% | st_rad | 0.29% |
| st_teff | 0.19% | st_mass | 0.12% |

Every predictor is under 5% missing, so no column needed to be dropped, and dropping rows would only have thrown away usable planets. I chose imputation. The baseline uses the **median** of each training column, because astronomical quantities such as mass, period and distance have extreme outliers that would drag a mean far from a typical planet. Imputing the test set with the training medians (never test statistics) keeps the evaluation honest.

As a comparison I repeated the whole pipeline with **mean** imputation (results in section 5). Orbital period and semi-major axis were log10-transformed because of the EDA findings above, and all features were standardized with the training mean and standard deviation. A column of ones was added for the intercept.

The data was split 80/20 with a fixed seed (np.random.default_rng(42)): 4,728 training rows and 1,183 test rows. The split happened before any median, mean or scaling statistics were computed, to avoid leakage.

## 4. Linear regression

I fitted the model two ways: the normal equation using np.linalg.pinv, and batch gradient descent (learning rate 0.05, 5,000 epochs). MAE, RMSE and $R^2$ were written by hand in NumPy.

## 5. Results

| Method | Intercept | MAE (K) | RMSE (K) | $R^2$ |
|---|---|---|---|---|
| Normal equation, median imputation | 835.92 | 153.68 | 270.66 | 0.6755 |
| Gradient descent, median imputation | 835.92 | 153.68 | 270.66 | 0.6755 |
| Normal equation, mean imputation | 835.92 | 153.31 | 270.12 | 0.6768 |

The two solvers agree: the largest difference between their coefficients is about 6e-12, so gradient descent fully converged to the closed-form answer. The model explains about 67.5% of the variance on the test set. Mean imputation was marginally better than median ($R^2$ 0.6768 vs 0.6755, MAE 153.31 K vs 153.68 K). The difference is tiny, so the choice of imputation barely matters here. This is expected because the amount of missing data is so small. I kept median as the baseline for the reason given in section 3.

## Coefficients (standardized features)

| Feature | Weight (K per std. dev.) |
|---|---|
| log_orbper | -400.65 |
| log_orbsmax | -28.74 |
| sy_dist | +7.38 |
| st_rad | +41.28 |
| pl_bmasse | +76.97 |
| st_mass | +79.89 |
| pl_rade | +113.73 |
| st_teff | +126.32 |

Orbital period has by far the largest weight, and a longer period lowers the predicted temperature, as expected. Stellar temperature is the largest positive weight. The intercept of 835.92 K is the prediction for an average planet.

## 6. Hottest and coldest predicted planets (test set)

| | Planet | Predicted (K) | Actual (K) |
|---|---|---|---|
| Hottest | V2376 Ori b | 2,886.35 | 2,100.00 |
| Coldest | COCONUTS-2 b | -2,680.43 | 493.00 |

The coldest prediction is physically impossible (below 0 K), which is discussed next.

## 7. Limitations

**Negative temperatures.** The worst case is COCONUTS-2 b, a directly imaged planet on a very wide orbit (period of roughly a million years). Its actual Teq is 493 K, but the model predicts -2,680 K. A linear model has no lower bound, and the huge log_orbper value multiplied by -400.65 pushes the output far below zero. At the hot end, V2376 Ori b was predicted at 2,886 K against an actual 2,100 K (error +786 K).

**Wrong functional form.** Equilibrium temperature scales multiplicatively with stellar temperature and distance, but the model adds terms together. Predicting log10(Teq) instead, or using the blackbody estimate as a feature, would probably fix both the negative predictions and the curvature at high temperatures. I did not try this.

**Collinearity.** log_orbper and log_orbsmax carry nearly the same information, so the model puts almost all of the distance effect on log_orbper (-400.65) and very little on log_orbsmax (-28.74). The individual weights are therefore not very reliable on their own, although the predictions are still fine.

**Planet radius.** pl_rade gets a large positive weight (+113.73), but radius does not set equilibrium temperature. The likely reason is that hot Jupiters are both hot and puffed up, so radius picks up part of the irradiation signal.